

WHAT IS SEQUENTIAL EXECUTION?

The Foundation of Logic

Sequential execution means that instructions are executed one after another, in the exact order they are written, from top to bottom.

THE RECIPE ANALOGY

1. Preheat Oven
2. Mix Ingredients
3. Bake Cookies
4. Let Cool

Steps must happen in order; you cannot cool before you bake.

Aspect	Explanation
Predictability	You always know what will happen next.
Debugging	Trace through code line by line to find errors.
Foundation	All programs are built on this sequence.

THE THREE-STEP PATTERN

1. Input

Gather data from the user, files, or sensors.

2. Process

Perform calculations or transformations on that data.

3. Output

Display or save the finalized results.

```
# Greeting Program
name = input("Name? ")
msg = "Hello, " + name
print(msg)
```

```
# Area Calculator
w = float(input("Width: "))
h = float(input("Height: "))
area = w * h
print("Area:", area)
```


VARIABLES: STORING DATA

The Labeled Box

A variable is a labeled container for a value. You can **Put** (assign), **Look** (read), and **Change** (update) its contents.

```
x = 5          # Step 1: Assign
y = 10         # Step 2: Assign
sum = x + y    # Step 3: Compute
print(sum)    # Step 4: Output 15
```

Naming Rules

Rule	Correct	Incorrect
Letters/Underscore	count	1stVal
No Special Chars	user_2	user-2
Case Sensitive	Score	score
No Keywords	total	while

DATA TYPES: INFORMATION CATEGORIES

Type	Name	Examples	Operations
<code>int</code>	Integer	<code>5, -12</code>	Math (+, -, *, /)
<code>float</code>	Decimal	<code>3.14, 2.0</code>	Math (+, -, *, /)
<code>str</code>	String	<code>"Hello"</code>	Concatenation (+)
<code>bool</code>	Boolean	<code>True, False</code>	Logic (and, or)

Why Types Matter

Operations depend on type. You cannot divide text by a number. Memory and meaning change based on type: `"5"` is text, while `5` is a number.

```
# Type Conversion
age = "25" # String
num = int(age) + 1 # 26
msg = "Age: " + str(num)
```


COMMON PATTERNS

Variable Swap

```
temp = a
a = b
b = temp
```

Accumulator

```
total = 0
total = total + val
```

Counter

```
count = 0
count = count + 1
```

Building Strings

Appending text to a variable to create a final message.

```
msg = msg + "New Word"
```

Average Calculation

Summing multiple inputs and dividing by the total count.

```
avg = (n1 + n2 + n3) / 3
```

COMMON MISTAKES

Mistake	Explanation	Solution
Undefined Use	Calling a variable before it exists.	Assign values before usage.
Input Type	<code>input()</code> always returns a string.	Use <code>int()</code> or <code>float()</code> .
Mixed Types	Adding strings and integers directly.	Convert to <code>str()</code> first.
Integer Div	Unexpected rounding in division.	Use <code>/</code> for floats, <code>//</code> for ints.
Vague Names	Using <code>x</code> , <code>y</code> , <code>z</code> for everything.	Use descriptive names like <code>price</code> .

HANDS-ON: SHOPPING CART

The Workflow

- > **1. Identify Inputs:** Prices of three distinct items.
- > **2. Plan Processing:** Sum the prices, then calculate 8% tax.
- > **3. Plan Output:** Show subtotal, tax amount, and total.

Pro Tip: Always test with simple data (e.g., \$10, \$20, \$30) to verify your math.

```
# Shopping Cart Code
p1 = float(input("Item 1: "))
p2 = float(input("Item 2: "))
p3 = float(input("Item 3: "))

subtotal = p1 + p2 + p3
tax = subtotal * 0.08
total = subtotal + tax

print("Total: $", total)
```